Natural Language Processing & Word Embeddings

Pass • 100/100 points earned

You must earn 80/100 points to pass.

Re-submit

Question 1: 10/10 points

Suppose you download a pre-trained word embedding which has been trained on a huge corpus of text. You then use this word embedding to train an RNN for a language task of recognizing if someone is happy from a short snippet of text, using a small training set.

x (input text)	y (happy?)
I'm feeling wonderful today!	1
I'm bummed that my cat is ill.	0
Really enjoying this!	1

True/False: Then even if the word "upset" does not appear in your small training set, your RNN might reasonably be expected to recognize "I'm upset" as deserving a label y = 0.

- (•) True

Correct

Yes, word vectors empower your model with an incredible ability to generalize. The vector for "upset" would contain a negative/unhappy connotation which will probably make your model classify the sentence as a "0".

- ( ) False

Question 2: 10/10 points

You have trained word embeddings using a text dataset of t_1 words. You are considering using these word embeddings for a language task, for which you have a separate labeled dataset of t_2 words. Keeping in mind that using word embeddings is a form of transfer learning, under which of these circumstances would you expect the word embeddings to be helpful?

- [x] When t_1 is larger than t_2
- [ ] When t_1 is smaller than t_2

Correct
Transfer embeddings to new tasks with smaller training sets.

- [ ] When t_1 is equal to t_2

Question 3: 10/10 points

When learning word embeddings, words are automatically generated along with the surrounding words.

- [ ] True
- [x] False

Correct
We pick a given word and try to predict its surrounding words or vice versa.

Question 4: 10/10 points

True/False: Suppose you learn a word embedding for a vocabulary of 20000 words. Then the embedding vectors could be 1000 dimensional, so as to capture the full range of variation and meaning in those words.

- (selected) True

Correct
The dimension of word vectors is usually smaller than the size of the vocabulary. Most common sizes for word vectors range between 50 and 1000.

- False

Question 5: 10/10 points

What is t-SNE?

- A linear transformation that allows us to solve analogies on word vectors
- (selected) A non-linear dimensionality reduction technique

Correct
Yes

- A supervised learning algorithm for learning word embeddings
- An open-source sequence modeling library

Question 6: 10/10 points

Question 6: 10/10 points

Suppose you have a 10000 word vocabulary, and are learning 100-dimensional word embeddings. The word2vec model uses the following softmax function:

$$P(t \mid c) = \frac{e^{\theta_t^T e_C}}{\sum_{t'=1}^{10000} e^{\theta_t^T e_C}}$$

Which of these statements are correct? Check all that apply.

- [x] θ_t and e_c are both trained with an optimization algorithm.

  Correct
  To review this concept watch the *Word2Vec* lecture.

- [x] θ_t and e_c are both 100 dimensional vectors.
  Feedback: To review this concept watch the *Word2Vec* lecture.

  Correct

- [ ] θ_t and e_c are both 10000 dimensional vectors.

- [ ] After training, we should expect θ_t to be very close to e_c when *t* and *c* are the same word.

Question 7: 10/10 points

True/False: In the word2vec algorithm, you estimate *P(t | c)*, where *t* is the target word and *c* is a context word. *t* and *c* are chosen from the training set to be nearby words.

Question 7: 10/10 points

True/False: In the word2vec algorithm, you estimate *P(t | c)*, where *t* is the target word and *c* is a context word. *t* and *c* are chosen from the training set to be nearby words.

- True

Correct
Yes, *t* and *c* are chosen from the training set to be nearby words.

- False

Question 8: 10/10 points

Let A be an embedding matrix, and let o_{4567} be a one-hot vector corresponding to word 4567. Then to get the embedding of word 4567, why don't we call $A * o_{4567}$ in Python?

- The correct formula is $A^T * o_{4567}$.
- It is computationally wasteful.

Correct
Yes, the element-wise multiplication will be extremely inefficient.

- None of the answers are correct: calling the Python snippet as described above is fine.
- This doesn't handle unknown words (<UNK>).

Question 9: 10/10 points

Suppose you have a 10000 word vocabulary, and are learning 500-dimensional word embeddings. The GloVe model minimizes this objective:

$$\min \sum_{i=1}^{10,000} \sum_{j=1}^{10,000} f(X_{ij})(\theta_i^T e_j + b_i + b_j' - log X_{ij})^2$$

True/False: X_{ij} is the number of times word j appears in the context of word i.

- (•) True
- ( ) False

Correct
X_{ij} is the number of times word j appears in the context of word i.

Question 10: 10/10 points

Which of these equations do you think should hold for a good word embedding? (Check all that apply)

- [ ] $e_{man} - e_{woman} \approx e_{aunt} - e_{uncle}$
- [x] $e_{man} - e_{woman} \approx e_{uncle} - e_{aunt}$

Correct
The order of words is correct in this analogy.

- [x] $e_{man} - e_{uncle} \approx e_{woman} - e_{aunt}$
- [ ] $e_{man} - e_{aunt} \approx e_{woman} - e_{uncle}$

Correct
The order of words is correct in this analogy.